1. Introduction to SQL Triggers

Summary: *In this tutorial, you will have an overview of database trigger, its advantages and disadvantages.*

SQL trigger is an SQL statements or a set of SQL statements which is stored to be activated or fired when an event associating with a database table occurs. The event can be any event including INSERT, UPDATE and DELETE.

Sometimes a trigger is referred as a special kind of stored procedure in term of procedural code inside its body. The difference between a trigger and a stored procedure is that a trigger is activated or called when an event happens in a database table, a stored procedure must be called explicitly. For example you can have some business logic to do before or after inserting a new record in a database table.

Before applying trigger in your database project, you should know its pros and cons to use it properly.

Advantages of using SQL trigger

- SQL Trigger provides an alternative way to check integrity.
- SQL trigger can catch the errors in business logic in the database level.
- SQL trigger provides an alternative way to run scheduled tasks. With SQL trigger, you don't have to wait to run the scheduled tasks. You can handle those tasks before or after changes being made to database tables.
- SQL trigger is very useful when you use it to audit the changes of data in a database table.

Disadvantages of using SQL trigger

- SQL trigger only can provide extended validation and cannot replace all the validations. Some simple validations can be done in the application level. For example, you can validate input check in the client side by using javascript or in the server side by server script using PHP or ASP.NET.
- SQL Triggers executes invisibly from client-application which connects to the database server so it is difficult to figure out what happen underlying database layer.
- SQL Triggers run every updates made to the table therefore it adds workload to the database and cause system runs slower.

Triggers or stored procedures? It depends on the the situation but it is practical that if you have no way to get the work done with stored procedure, think about triggers.

2. Trigger Implementation in MySQL

MySQL finally supports one of the most important features of an enterprise database server which is called trigger since version 5.0.2. Trigger is implemented in MySQL by following the syntax of standard SQL:2003. When you create a trigger in MySQL, its definition stores in the file with extension .TRG in a database folder with specific name as follows: */data_folder/database_name/table_name.trg*

The file is in plain text format so you can use any plain text editor to modify it.

While trigger is implemented in MySQL has all features in standard SQL but there are some restrictions you should be aware of like following:

- It is not allowed to call a stored procedure in a trigger.
- It is not allowed to create a trigger for views or temporary table.
- It is not allowed to use transaction in a trigger.

- Return statement is disallowed in a trigger.
- Creating a trigger for a database table causes the query cache invalidated. Query cache allows you to store the result of query and corresponding select statement. In the next time, when the same select statement comes to the database server, the database server will use the result which stored in the memory instead of parsing and executing the query again.
- All trigger for a database table must have unique name. It is allowed that triggers for different tables having the same name but it is recommended that trigger should have unique name in a specific database. To create the trigger, you can use the following naming convention: (BEFORE | AFTER)_tableName_(INSERT| UPDATE | DELETE)

3. Create the First Trigger in MySQL

Summary: *In this tutorial, you will learn how to create the first **trigger in MySQL** by using **CREATE TRIGGER** statement.*
You should follow the previous tutorials on triggers such as Introduction to SQL Triggers and Trigger Implementation in MySQL first before reading this tutorial.
Let's start creating the first trigger in MySQL by following a simple scenario. In the sample database, we have table *employees* as follows:

```
01 CREATE TABLE `employees` (
02  `employeeNumber` int(11) NOT NULL,
03  `lastName` varchar(50) NOT NULL
,
04  `firstName` varchar(50) NOT NULL,
05  `extension` varchar(10) NOT NULL,
06  `email` varchar(100) NOT NULL,
07  `officeCode` varchar(10) NOT NULL,
08  `reportsTo` int(11) default NULL,
09   `jobTitle` varchar(50) NOT NULL,
10   PRIMARY KEY  (`employeeNumber`)
11 )
```

Now you want to keep the changes of employee's data in another table whenever data of an employee's record changed. In order to do so you create a new table called *employees_audit* to keep track the changes.

```
1 CREATE TABLE employees_audit (
2 id int(11) NOT NULL AUTO_INCREMENT,
3 employeeNumber int(11) NOT NULL,
4 lastname varchar(50) NOT NULL,
5
6 changedone datetime DEFAULT NULL,
7 action varchar(50) DEFAULT NULL,
8 PRIMARY KEY (id)
```

```
9 )
```

In order to keep track the changes of last name of employee we can create a trigger that is fired before we make any update on the *employees* table. Here is the source code of the trigger

```
01 DELIMITER $$
02 CREATE TRIGGER before_employee_update
03 BEFORE UPDATE ON employees
04 FOR EACH ROW BEGIN
05 INSERT INTO employees_audit
06 SET action = 'update',
07 employeeNumber = OLD.employeeNumber,
08 lastname = OLD.lastname,
09 changedon = NOW(); END$$
10 DELIMITER ;
```

You can test the trigger which created by updating last name of any employee in *employees*table. Suppose we update last name of employee which has employee number is 3:

```
1 UPDATE employees
2 SET lastName = 'Phan'
3 WHERE employeeNumber = 1056
```

Now when you can see the changes audited automatically in the *employees_audit* table by executing the following query

```
1 SELECT *
2 FROM employees_audit
```

In order to create a trigger you use the following syntax:

```
1 CREATE TRIGGER trigger_name trigger_time trigger_event
2 ON table_name
3 FOR EACH ROW
4 BEGIN
5 ...
6 END
```

- CREATE TRIGGER statement is used to create triggers.
- The trigger name should follow the naming convention [trigger time]_[table name]_[trigger event], for example *before_employees_update*
- Trigger activation time can be BEFORE or AFTER. You must specify the activation time when you define a trigger. You use BEFORE when you want to process action prior to the change being made in the table and AFTER if you need to process action after changes are made.
- Trigger event can be INSERT, UPDATE and DELETE. These events cause trigger to fire and process logic inside trigger body. A trigger only can fire with one event. To define trigger which are fired by multiple events, you have to define multiple triggers, one for each event. Be noted that any SQL statements make update data in database table will cause trigger to fire. For example, LOAD DATA statement insert records into a table will also cause the trigger associated with that table to fire.
- A trigger must be associated with a specific table. Without a table trigger does not exist so you have to specify the table name after the ON keyword.

- You can write the logic between BEGIN and END block of the trigger.
- MySQL gives you OLD and NEW keyword to help you write trigger more efficient. The OLD keyword refers to the existing row before you update data and the NEW keyword refers to the new row after you update data.

In this tutorial, you have learned how to create the first trigger in MySQL. You've written the first trigger to audit changes of last name of employee in *employees* table.

4. Managing Trigger in MySQL

Summary: *In this tutorial, you will learn how to manage triggers in MySQL including listing all triggers in a given database or removing triggers.*

Once created trigger associated with a table, you can view the trigger by going directly to the folder which contains the trigger. Trigger is stored as plain text file in the database folder as follows: */data_folder/database_name/table_name.trg,* with any plain text editor such as notepad you can view it. MySQL provides you another way to view the trigger by executing SQL statement:

```
1 SELECT * FROM Information_Schema.Trigger
2 WHERE Trigger_schema = 'database_name' AND
3     Trigger_name = 'trigger_name';
```

In this method you are not only view the content of the trigger but also other metadata associating with it such as table name, definer (name of MySQL who created the trigger).

If you want to retrieve all triggers associated with a database just executing the following SQL statement:

```
1 SELECT * FROM Information_Schema.Trigger
2 WHERE Trigger_schema = 'database_name';
```

To find all triggers associating with a database table, just executing the following SQL statement:

```
1 SELECT * FROM Information_Schema.Trigger
2 WHERE Trigger_schema = 'database_name' AND
3     Event_object_table = 'table_name';
```

In MySQL you are not only able to view the trigger but also remove an existing trigger. To remove a trigger you can use the SQL statement DROP TRIGGER as follows:

```
1 DROP TRIGGER table_name.trigger_name
```

For example if you want to drop trigger *before_employees_update* which associated with the table *employees,* you can perform the following query:

```
1 DROP TRIGGER employees.before_employees_update
```

To modify a trigger, you have to delete it first and recreate it. MySQL doesn't provide you SQL statement to alter an existing trigger like altering other database objects such as tables or stored procedures.

In this tutorial, you've learned how to view and retrieve information related to a trigger. You've also learned how to delete an existing trigger by using DROP TRIGGER statement.